

SMART CONTRACT VM & COMPILER

AEGIS PLATFORM

# Aegis PQSynQ

SynQ VM & Compiler Post-Quantum Cryptography Facade

| MODULE       | TYPE             | STATUS     |
|--------------|------------------|------------|
| aegis-pqsynq | PQC Facade Crate | Production |

#### LEGAL & PROPRIETARY NOTICE

Aegis PQSynQ is a proprietary post-quantum cryptography facade for the SynQ VM and compiler ecosystem. SynQ is Synergy Engineering Labs' own smart contract VM and compiler language. This module provides the cryptographic backbone for SynQ programs. All algorithms are NIST-standardized. Unauthorized reproduction, distribution, or use is strictly prohibited. This manual is confidential and intended for authorized SynQ developers only.

## Table of Contents

|   |                                        |    |
|---|----------------------------------------|----|
| 1 | Introduction & SynQ Ecosystem          | 5  |
| 2 | Architecture & Facade Pattern          | 12 |
| 3 | Algorithm Support & Configuration      | 19 |
| 4 | Dependency Setup & Integration         | 26 |
| 5 | KEM & Signature API Reference          | 33 |
| 6 | Trait-Based Abstraction & Factories    | 40 |
| 7 | Domain Separation & Contextual Signing | 47 |
| 8 | SynQ Smart Contract Patterns           | 54 |
| 9 | Testing, Validation & Deployment       | 61 |

# Introduction & SynQ Ecosystem



## Post-Quantum Smart Contracts

Aegis PQSynQ is the cryptographic backbone of the SynQ smart contract VM.

Aegis PQSynQ is a specialized post-quantum cryptography facade crate designed for the SynQ VM and compiler ecosystem. SynQ is Synergy Engineering Labs' own smart contract virtual machine and domain-specific language, analogous to EVM for Ethereum or Move for Aptos. PQSynQ provides all NIST-standardized post-quantum algorithm families through a clean, trait-based API that SynQ programs use for transaction signing, smart contract attestation, quantum-safe channel establishment, and verifiable randomness.

## Key Features

- ML-KEM-512/768/1024 (key encapsulation) ML-DSA-44/65/87 (digital signatures)
- HQC-KEM-128/192/256 (alternative KEM) FN-DSA-512/1024 (alternative signature)
- Contextual signing with domain separation wasm32 compatible (`--no-default-features`)
- Zero-copy trait-based abstractions

## Canonical Import

```
// In your SynQ Cargo.toml
[dependencies]
pqsynq = { package = "aegis-pqsynq", path = "../aegis-pqsynq/pqsynq" }
```



SynQ programs use `aegis-pqsynq` to sign transactions (ensuring only the correct sender can execute), verify smart contract state transitions (preventing double-spending or invalid state changes), establish quantum-safe channels between VM participants, and generate verifiable randomness for protocol operations. All cryptographic operations are deterministic and auditable.

## SECTION 2

# Architecture & Facade Pattern



## Trait-Based Abstraction

Clean API hides algorithm complexity; swap implementations without breaking contracts.

## Two Core Traits

PQSynQ exposes two main traits: `KeyEncapsulation` (KEM) and `DigitalSignature`. All algorithms implement these traits:

```
pub trait KeyEncapsulation {
    fn keygen(&mut self) -> Result<(Vec, Vec)>;
    fn encapsulate(&mut self, pk: &[u8]) -> Result<(Vec, Vec)>;
    fn decapsulate(&mut self, ct: &[u8], sk: &[u8]) -> Result>;
}

pub trait DigitalSignature {
    fn keygen(&mut self) -> Result<(Vec, Vec)>;
    fn detached_sign(&mut self, msg: &[u8], sk: &[u8]) -> Result>;
    fn verify_detached(&mut self, msg: &[u8], sig: &[u8], pk: &[u8]) -> Result>;
    fn sign_ctx(&mut self, msg: &[u8], sk: &[u8], ctx: &[u8]) -> Result>;
}
```

```
fn verify_ctx(&mut self, msg: &[u8], sig: &[u8], pk: &[u8], ctx: &[u8]) -> Result<bool, Error> {  
}
```

## Algorithm Factories

Create algorithm instances via factory functions:

```
let kem = Kem::mlkem512(); // ML-KEM-512  
let kem = Kem::mlkem768(); // ML-KEM-768  
let kem = Kem::hqckem128(); // HQC-KEM-128  
  
let sig = Sign::mldsa44(); // ML-DSA-44  
let sig = Sign::mldsa65(); // ML-DSA-65  
let sig = Sign::fndsa512(); // FN-DSA-512
```



### Trait Abstraction

Algorithms implement standard traits; contracts don't care about implementation details.



### Pluggable Implementations

Swap algorithms by selecting different factory; no contract code changes.



### Zero-Copy API

Operates on byte slices; minimal allocation overhead.

# Algorithm Support & Configuration



## Six NIST Families for SynQ

Complete algorithm coverage with feature-gated optional support.

### Default Algorithms

| Category  | Algorithm                             | Feature Flag | Default  |
|-----------|---------------------------------------|--------------|----------|
| KEM       | ML-KEM-512, ML-KEM-768, ML-KEM-1024   | mlkem        | Enabled  |
| KEM       | HQC-KEM-128, HQC-KEM-192, HQC-KEM-256 | hqckem       | Optional |
| Signature | ML-DSA-44, ML-DSA-65, ML-DSA-87       | mldsa        | Enabled  |
| Signature | FN-DSA-512, FN-DSA-1024               | fndsa        | Enabled  |

### Feature Configuration

```
// Cargo.toml: Default features (std, mlkem, mldsa, fndsa)
pqsynq = { package = "aegis-pqsynq", path = "../aegis-pqsynq/pqsynq" }

// Minimal build (no_std, only ML-KEM)
pqsynq = { package = "aegis-pqsynq", path = "../aegis-pqsynq/pqsynq", default-features = false }

// Full support (all algorithms)
pqsynq = { package = "aegis-pqsynq", path = "../aegis-pqsynq/pqsynq", features = ["std", "mlkem", "mldsa", "fndsa"] }

// wasm32 build
pqsynq = { package = "aegis-pqsynq", path = "../aegis-pqsynq/pqsynq", default-features = false }
```

## Validation Gates (All Passing)

- `cargo fmt --all --check` `cargo clippy -p aegis-pqsynq --all-targets --all-features -- -D warnings`
- `cargo test -p aegis-pqsynq --all-features --all-targets`
- `cargo check -p aegis-pqsynq --no-default-features`
- `cargo check -p aegis-pqsynq --target wasm32-unknown-unknown --no-default-features`



### Recommended for SynQ Contracts

Use default features (std + mlkem + mldsa + fndsa) for full flexibility. For constrained embedded SynQ runtimes, disable std and select only necessary algorithms.

## SECTION 4

# Dependency Setup & Integration



## Add PQSynQ to Your SynQ Project

One-step integration into your smart contract VM.

## Integration Steps

1

### Add to Cargo.toml

Add the canonical import under [dependencies].

2

### Import in Your Code

```
use pqsynq::{Kem, Sign};
```

3

### Create Algorithm Instances

Use factory functions to instantiate specific algorithms.

#### 4 Perform Cryptographic Operations

Call `keygen`, `sign`, `verify`, `encapsulate`, `decapsulate` via trait methods.

### Minimal SynQ Contract Example

```
use pqsynq::{Kem, Sign};

pub contract QuantumSafeVault {
    fn setup(&mut self) {
        // Generate signature keypair for contract owner
        let mut signer = Sign::mldsa44();
        let (vk, sk) = signer.keygen().unwrap();

        // Store public key for later verification
        self.owner_pubkey = vk;
    }

    pub fn withdraw(
        &mut self,
        amount: u64,
        signature: &[u8],
        owner_sk: &[u8],
    ) -> Result {
        // Verify withdrawal is signed by owner
        let mut signer = Sign::mldsa44();
        let msg = format!("withdraw:{}", amount).as_bytes().to_vec();

        let is_valid = signer.verify_detached(
            &msg,
            signature,
            &self.owner_pubkey
        )?;

        if !is_valid {
            return Err("Invalid signature");
        }

        // Process withdrawal
        self.balance -= amount;
    }
}
```



```
    Ok(())  
  }  
}
```

## SECTION 5

# KEM & Signature API Reference

---



## Complete Function Reference

All public API functions documented with examples.

## Key Encapsulation (KEM) Example

```
use pqsynq::{Kem, KeyEncapsulation};  
  
fn kem_roundtrip() -> Result<()> {  
    let mut kem = Kem::mlkem512();  
  
    // Receiver: generate keypair  
    let (pk, sk) = kem.keygen()?;  
  
    // Sender: encapsulate to receiver's public key  
    let (ciphertext, shared_secret_sender) = kem.encapsulate(&pk)?;  
  
    // Receiver: decapsulate ciphertext to recover shared secret  
    let shared_secret_receiver = kem.decapsulate(&ciphertext, &sk)?;  
  
    // Verify both sides recovered same secret  
    assert_eq!(shared_secret_sender, shared_secret_receiver);  
}
```

```
    Ok(())  
}
```

## Digital Signature Example

```
use pqsynq::{Sign, DigitalSignature};  
  
fn signature_roundtrip() -> Result<()> {  
    let mut signer = Sign::mldsa44();  
  
    // Signer: generate keypair  
    let (vk, sk) = signer.keygen()?;  
  
    // Signer: detached sign a message  
    let message = b"Important transaction";  
    let signature = signer.detached_sign(message, &sk)?;  
  
    // Verifier: verify signature  
    let is_valid = signer.verify_detached(message, &signature, &vk)?;  
    assert!(is_valid);  
  
    Ok(())  
}
```

## Error Type

```
pub enum PqcError {  
    KeygenFailed,  
    EncapFailed,  
    DecapFailed,  
    SignFailed,  
    VerifyFailed,  
    NotImplemented,           // For unsupported operations  
    InvalidInput,  
}
```

# Trait-Based Abstraction & Factories



## Composable Cryptography

Switch algorithms without changing contract logic.

### KEM Factories

```
impl Kem {
  pub fn mlkem512() -> Self { /* ... */ }
  pub fn mlkem768() -> Self { /* ... */ }
  pub fn mlkem1024() -> Self { /* ... */ }
  pub fn hqckem128() -> Self { /* ... */ }
  pub fn hqckem192() -> Self { /* ... */ }
  pub fn hqckem256() -> Self { /* ... */ }
}
```

### Signature Factories

```
impl Sign {
  pub fn mldsa44() -> Self { /* ... */ }
  pub fn mldsa65() -> Self { /* ... */ }
  pub fn mldsa87() -> Self { /* ... */ }
  pub fn fndsa512() -> Self { /* ... */ }
  pub fn fndsa1024() -> Self { /* ... */ }
}
```



#### Algorithm Selection in SynQ

For most SynQ contracts, ML-KEM-512 (smallest KEM) and ML-DSA-44 (smallest signature) provide excellent post-quantum security with minimal overhead. Use

larger variants only if your threat model requires 256+ bits of effective security.

## SECTION 7

# Domain Separation & Contextual Signing



## Prevent Signature Reuse Across Contexts

Domain separation ensures signatures are bound to specific SynQ contract types.

## Contextual Signature API

ML-DSA supports `sign_ctx(msg, sk, context)` and `verify_ctx(msg, sig, pk, context)`. This prevents signature replay attacks across different contract types.

```
use pqsynq::{Sign, DigitalSignature};

// Contract A: Payment transfer
let mut signer = Sign::mldsa44();
let msg = b"transfer:100";
let ctx = b"synq-contract-payment-v1"; // Domain separation
let sig_a = signer.sign_ctx(msg, &sk, ctx)?;

// Contract B: Voting (different context)
let ctx_b = b"synq-contract-voting-v1"; // Different domain
let sig_b = signer.sign_ctx(msg, &sk, ctx_b)?;

// sig_a cannot be used in voting context (and vice versa)
let valid = signer.verify_ctx(msg, &sig_a, &vk, ctx_b)?; // Returns false
```

## FN-DSA Note

FN-DSA does not support contextual signing. If you use FN-DSA, you must implement domain separation manually by prepending context to the message:

```
let mut signer = Sign::fndsa512();
let ctx = b"synq-contract-type-v1";
let msg_with_ctx = [ctx.as_slice(), msg].concat();
let sig = signer.detached_sign(&msg_with_ctx, &sk)?;
```

### ✓ Domain Separation Best Practice

Always use context-aware signatures (sign\_ctx/verify\_ctx) when available. If using FN-DSA, prepend a context identifier to all messages. This prevents signature reuse across different SynQ contract types.

## SECTION 8

# SynQ Smart Contract Patterns



## Common PQC Usage Patterns

Real-world smart contract examples using aegis-pqsynq.

## Pattern 1: Transaction Signature Verification

```
pub contract PQWallet {
  pub fn execute_transaction(
    &mut self,
    recipient: Address,
```

```

        amount: u64,
        signature: &[u8],
    ) -> Result<()> {
        // Only owner can execute (verified via ML-DSA signature)
        let mut signer = Sign::mldsa44();
        let tx_msg = format!("tx:{{}}:{{}}", recipient, amount);

        let is_valid = signer.sign_ctx(
            tx_msg.as_bytes(),
            &signature,
            &self.owner_pubkey,
            b"synq-wallet-transaction-v1"
        )?;

        if !is_valid {
            return Err("Signature invalid");
        }

        self.balance -= amount;
        emit_transfer(recipient, amount);
        Ok(())
    }
}

```

## Pattern 2: Quantum-Safe Session Establishment

```

pub contract PQChannel {
    pub fn initiate_handshake(
        &mut self,
        peer_ml_kem_pk: &[u8],
        peer_signature_pk: &[u8],
    ) -> Result<> {
        // Establish quantum-safe session via ML-KEM key exchange
        let mut kem = Kem::mlkem512();
        let (ciphertext, session_secret) = kem.encapsulate(peer_ml_kem_pk)?;

        // Sign ciphertext to prove ownership of this contract
        let mut signer = Sign::mldsa44();
        let sig = signer.sign_ctx(
            &ciphertext,
            &self.my_signature_sk,

```

```

        b"synq-channel-handshake-v1"
    )?;

    // Store session secret for future encrypted communication
    self.session_key = kdf_session(&session_secret)?;

    Ok([ciphertext, sig].concat())
}
}

```

## Pattern 3: Attestation & Proof

```

pub contract PQAttestor {
    pub fn issue_attestation(
        &mut self,
        claim: &[u8],
    ) -> Result {
        // Attestor signs claim with ML-DSA-44
        let mut signer = Sign::mldsa44();
        let attestation = signer.sign_ctx(
            claim,
            &self.attestor_sk,
            b"synq-attestation-v1"
        )?;

        Ok(attestation)
    }

    pub fn verify_attestation(
        &mut self,
        claim: &[u8],
        attestation: &[u8],
        attestor_pk: &[u8],
    ) -> Result {
        let mut signer = Sign::mldsa44();
        signer.verify_ctx(
            claim,
            attestation,
            attestor_pk,
            b"synq-attestation-v1"
        )
    }
}

```

```
}  
}
```

## SECTION 9

# Testing, Validation & Deployment



## Production-Ready Validation

All strict gates passing; ready for smart contract deployment.

## Build Validation Commands

### 1 Format Check

```
cargo fmt -p aegis-pqsynq --all --check
```

### 2 Linting (Strict)

```
cargo clippy -p aegis-pqsynq --all-targets --all-features --no-deps -- -D  
warnings
```

### 3 Unit Tests

```
cargo test -p aegis-pqsynq --all-features --all-targets --locked
```

### 4 no\_std Check

```
cargo check -p aegis-pqsynq --no-default-features --locked
```

### 5 wasm32 Check

```
cargo check -p aegis-pqsynq --target wasm32-unknown-unknown --no-default-  
features --locked
```



## NIST KAT Validation

All algorithms validated against NIST Known-Answer Test vectors in [5-nist-kat-vectors/](#).  
Before deployment, verify:

- ML-KEM-512/768/1024 keygen, encapsulate, decapsulate
- ML-DSA-44/65/87 keygen, sign, verify    HQC-KEM-128/192/256 operations
- FN-DSA-512/1024 operations

## Smart Contract Testing Checklist

### 1 Unit Tests on Algorithms

Test keygen, sign, verify, encapsulate, decapsulate independently.

### 2 Integration Tests in SynQ Contract

Test full transaction signing and verification flow.

### 3 Domain Separation Testing

Verify signatures fail when context is changed.

### 4 Error Handling

Ensure invalid inputs return appropriate PqcError variants.

### 5 Performance Profiling

Measure signature/KEM operation times on target hardware.

### 6 Audit & Code Review

Have cryptography expert review contract PQC usage patterns.

## Documentation Resources

- USER\_MANUAL.md — Full API reference
- CHANGELOG.md — Version history
- PRODUCTION\_CHECKLIST.md — Pre-deployment verification
- 5-nist-kat-vectors/ — Reference test vectors
- 3-benchmark-tests/ — Performance baseline



### SynQ Contract Deployment Checklist

All build gates passing (fmt, clippy -D warnings, tests, no\_std, wasm32). NIST KAT vectors validated. Smart contract unit and integration tests passing. Domain separation correctly implemented. Error handling complete. Ready for mainnet deployment.

**Aegis PQSynQ** | SynQ VM & Compiler Post-Quantum Cryptography Facade

© 2026 **Synergy Engineering Labs** | Patent Pending

Document Version 1.0 | Module Status: Production Ready